Module 31

Application Design and Development/1: Architecture

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

March 17, 2026

Week Recap

- Studied the Normal Forms and their Importance in Relational Design - how progressive increase of constraints can minimize redundancy in a schema

Week Recap

- Studied the Normal Forms and their Importance in Relational Design - how progressive increase of constraints can minimize redundancy in a schema
- Learnt how to decompose a schema into 3NF while preserving dependency and lossless join

Week Recap

- Studied the Normal Forms and their Importance in Relational Design - how progressive increase of constraints can minimize redundancy in a schema
- Learnt how to decompose a schema into 3NF while preserving dependency and lossless join
- Learnt how to decompose a schema into BCNF with lossless join

Week Recap

- Studied the Normal Forms and their Importance in Relational Design - how progressive increase of constraints can minimize redundancy in a schema
- Learnt how to decompose a schema into 3NF while preserving dependency and lossless join
- Learnt how to decompose a schema into BCNF with lossless join
- Using the specification for a Library Information System, we have illustrated how a schema can be designed and then refined for finalization

Week Recap

- Studied the Normal Forms and their Importance in Relational Design - how progressive increase of constraints can minimize redundancy in a schema
- Learnt how to decompose a schema into 3NF while preserving dependency and lossless join
- Learnt how to decompose a schema into BCNF with lossless join
- Using the specification for a Library Information System, we have illustrated how a schema can be designed and then refined for finalization
- Coding of various queries based on these schema are left as exercises

Week Recap

- Studied the Normal Forms and their Importance in Relational Design - how progressive increase of constraints can minimize redundancy in a schema
- Learnt how to decompose a schema into 3NF while preserving dependency and lossless join
- Learnt how to decompose a schema into BCNF with lossless join
- Using the specification for a Library Information System, we have illustrated how a schema can be designed and then refined for finalization
- Coding of various queries based on these schema are left as exercises
- Understood multi-valued dependencies to handle attributes that can have multiple values

Week Recap

- Studied the Normal Forms and their Importance in Relational Design - how progressive increase of constraints can minimize redundancy in a schema
- Learnt how to decompose a schema into 3NF while preserving dependency and lossless join
- Learnt how to decompose a schema into BCNF with lossless join
- Using the specification for a Library Information System, we have illustrated how a schema can be designed and then refined for finalization
- Coding of various queries based on these schema are left as exercises
- Understood multi-valued dependencies to handle attributes that can have multiple values
- Learnt Fourth Normal Form and decomposition to 4NF

Week Recap

- Studied the Normal Forms and their Importance in Relational Design - how progressive increase of constraints can minimize redundancy in a schema
- Learnt how to decompose a schema into 3NF while preserving dependency and lossless join
- Learnt how to decompose a schema into BCNF with lossless join
- Using the specification for a Library Information System, we have illustrated how a schema can be designed and then refined for finalization
- Coding of various queries based on these schema are left as exercises
- Understood multi-valued dependencies to handle attributes that can have multiple values
- Learnt Fourth Normal Form and decomposition to 4NF
- Discussed aspects of the database design process

Week Recap

- Studied the Normal Forms and their Importance in Relational Design - how progressive increase of constraints can minimize redundancy in a schema
- Learnt how to decompose a schema into 3NF while preserving dependency and lossless join
- Learnt how to decompose a schema into BCNF with lossless join
- Using the specification for a Library Information System, we have illustrated how a schema can be designed and then refined for finalization
- Coding of various queries based on these schema are left as exercises
- Understood multi-valued dependencies to handle attributes that can have multiple values
- Learnt Fourth Normal Form and decomposition to 4NF
- Discussed aspects of the database design process
- Studied the issues with temporal data

Module Objectives

Goals

- What are the Application Programs across various sectors?

Module Objectives

Goals

- What are the Application Programs across various sectors?
- Commonality of architecture across applications

Module Objectives

Goals

- What are the Application Programs across various sectors?
- Commonality of architecture across applications
- Understanding the classification and evolution of the architectures

Module Objectives

Goals

- What are the Application Programs across various sectors?
- Commonality of architecture across applications
- Understanding the classification and evolution of the architectures
- A look at the architecture for a few sample applications

Module Outline

- Application Programs

Module Outline

- Application Programs
- Application Architecture with classification and evolution

Module Outline

- Application Programs
- Application Architecture with classification and evolution
- Sample application architectures

Application Programs and Architecture

Application Programs: Internet / Web or Mobile

- **Financial:**

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,
  - Transportation: Uber, Ola Cab, Mega Cab, Meru Cab,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,
  - Transportation: Uber, Ola Cab, Mega Cab, Meru Cab,
  - Accommodation: Booking, OYO, AirBnb, Fabhotels, Treebo,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,
  - Transportation: Uber, Ola Cab, Mega Cab, Meru Cab,
  - Accommodation: Booking, OYO, AirBnb, Fabhotels, Treebo,
  - Navigation: Google Maps, MapQuest, Apple Maps,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,
  - Transportation: Uber, Ola Cab, Mega Cab, Meru Cab,
  - Accommodation: Booking, OYO, AirBnb, Fabhotels, Treebo,
  - Navigation: Google Maps, MapQuest, Apple Maps,
  - Food & Delivery: Zomato, Swiggy, UberEats, Dunzo,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,
  - Transportation: Uber, Ola Cab, Mega Cab, Meru Cab,
  - Accommodation: Booking, OYO, AirBnb, Fabhotels, Treebo,
  - Navigation: Google Maps, MapQuest, Apple Maps,
  - Food & Delivery: Zomato, Swiggy, UberEats, Dunzo,
- **Communication:**

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,
  - Transportation: Uber, Ola Cab, Mega Cab, Meru Cab,
  - Accommodation: Booking, OYO, AirBnb, Fabhotels, Treebo,
  - Navigation: Google Maps, MapQuest, Apple Maps,
  - Food & Delivery: Zomato, Swiggy, UberEats, Dunzo,
- **Communication:**
  - Live Interaction: Zoom, Google Meet, Teams, Webex, Skype,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,
  - Transportation: Uber, Ola Cab, Mega Cab, Meru Cab,
  - Accommodation: Booking, OYO, AirBnb, Fabhotels, Treebo,
  - Navigation: Google Maps, MapQuest, Apple Maps,
  - Food & Delivery: Zomato, Swiggy, UberEats, Dunzo,
- **Communication:**
  - Live Interaction: Zoom, Google Meet, Teams, Webex, Skype,
  - Mail: Gmail, Yahoo, Hotmail, Rediffmail, Enterprise Mail,

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,
  - Transportation: Uber, Ola Cab, Mega Cab, Meru Cab,
  - Accommodation: Booking, OYO, AirBnb, Fabhotels, Treebo,
  - Navigation: Google Maps, MapQuest, Apple Maps,
  - Food & Delivery: Zomato, Swiggy, UberEats, Dunzo,
- **Communication:**
  - Live Interaction: Zoom, Google Meet, Teams, Webex, Skype,
  - Mail: Gmail, Yahoo, Hotmail, Rediffmail, Enterprise Mail,
  - Intermittent Interaction: WhatsApp, Telegram, Signal, Skype

Application Programs: Internet / Web or Mobile

- **Financial:**
  - Netbanking: SBI, PNB, BoB, Canara, HDFC, ICICI
  - Insurance & Investment: LICI, PolicyBazaar, NSDL, NPS,
  - Share Market: ICICIDirect, Sharekhan, HDFCDirect
  - Payment Gateway: Paytm, GPay, Bhim UPI, PhonePe,
  - e-Commerce: Amazon, Flipkart, eBay, BigBazaar, BigBasket,
- **Travel & Tourism:**
  - Travel Reservations: IRCTC, Airlines, MakeMyTrip, Yatra,
  - Transportation: Uber, Ola Cab, Mega Cab, Meru Cab,
  - Accommodation: Booking, OYO, AirBnb, Fabhotels, Treebo,
  - Navigation: Google Maps, MapQuest, Apple Maps,
  - Food & Delivery: Zomato, Swiggy, UberEats, Dunzo,
- **Communication:**
  - Live Interaction: Zoom, Google Meet, Teams, Webex, Skype,
  - Mail: Gmail, Yahoo, Hotmail, Rediffmail, Enterprise Mail,
  - Intermittent Interaction: WhatsApp, Telegram, Signal, Skype
  - Social Media: Facebook, Instagram, Twitter, YouTube,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**
  - Issue Tracking: JIRA, BugZilla, Githubs, Gitlab,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**
  - Issue Tracking: JIRA, BugZilla, Githubs, Gitlab,
  - Online IDE: OnlineGDB, Codechef, Ideone,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**
  - Issue Tracking: JIRA, BugZilla, Githubs, Gitlab,
  - Online IDE: OnlineGDB, Codechef, Ideone,
  - VCS: Githubs, Gitlab, BitBucket, SourceForge,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**
  - Issue Tracking: JIRA, BugZilla, Githubs, Gitlab,
  - Online IDE: OnlineGDB, Codechef, Ideone,
  - VCS: Githubs, Gitlab, BitBucket, SourceForge,
- **Library:**

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**
  - Issue Tracking: JIRA, BugZilla, Githubs, Gitlab,
  - Online IDE: OnlineGDB, Codechef, Ideone,
  - VCS: Githubs, Gitlab, BitBucket, SourceForge,
- **Library:**
  - Digital Library: National Digital Library of India,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**
  - Issue Tracking: JIRA, BugZilla, Githubs, Gitlab,
  - Online IDE: OnlineGDB, Codechef, Ideone,
  - VCS: Githubs, Gitlab, BitBucket, SourceForge,
- **Library:**
  - Digital Library: National Digital Library of India,
  - Archives: Internet Archive, arXiv, Nextpoint,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**
  - Issue Tracking: JIRA, BugZilla, Githubs, Gitlab,
  - Online IDE: OnlineGDB, Codechef, Ideone,
  - VCS: Githubs, Gitlab, BitBucket, SourceForge,
- **Library:**
  - Digital Library: National Digital Library of India,
  - Archives: Internet Archive, arXiv, Nextpoint,
- **Education:**

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**
  - Issue Tracking: JIRA, BugZilla, Githubs, Gitlab,
  - Online IDE: OnlineGDB, Codechef, Ideone,
  - VCS: Githubs, Gitlab, BitBucket, SourceForge,
- **Library:**
  - Digital Library: National Digital Library of India,
  - Archives: Internet Archive, arXiv, Nextpoint,
- **Education:**
  - eLearning: BYJU's, IGNOU, NIIT, Edukart,

Application Programs: Internet / Web or Mobile (cont.)

- **Knowledge Discovery:**
  - Static: Google, Yahoo, Bing, Wikipedia, Encyclopedia.com,
  - Q&A: Quora, ASKfm, Yahoo Answers, Reddit, Digg,
- **Sports:**
  - Cricket: Cricbuzz, CricViz, Cricket-21, Cricket Exchange,
  - Tennis: ATP, ITF, SwingVision, TennisPAL, Tennis Clash,
- **Software Engineering:**
  - Issue Tracking: JIRA, BugZilla, Githubs, Gitlab,
  - Online IDE: OnlineGDB, Codechef, Ideone,
  - VCS: Githubs, Gitlab, BitBucket, SourceForge,
- **Library:**
  - Digital Library: National Digital Library of India,
  - Archives: Internet Archive, arXiv, Nextpoint,
- **Education:**
  - eLearning: BYJU's, IGNOU, NIIT, Edukart,
  - MOOCs: SWAYAM, edX, Coursera, Udemy,

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,
- **Health:**

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,
- **Health:**
  - Telemedicine: MDLIVE, Doctor on Demand,

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,
- **Health:**
  - Telemedicine: MDLIVE, Doctor on Demand,
  - National: Aarogy Setu, CoWin, NACO App,

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,
- **Health:**
  - Telemedicine: MDLIVE, Doctor on Demand,
  - National: Aarogy Setu, CoWin, NACO App,
- **Organizational ERP: (Intranet)**

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,
- **Health:**
  - Telemedicine: MDLIVE, Doctor on Demand,
  - National: Aarogy Setu, CoWin, NACO App,
- **Organizational ERP: (Intranet)**
  - Institutions: Students, Faculty, Course

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,
- **Health:**
  - Telemedicine: MDLIVE, Doctor on Demand,
  - National: Aarogy Setu, CoWin, NACO App,
- **Organizational ERP: (Intranet)**
  - Institutions: Students, Faculty, Course
  - Manufacturing: Suppliers, Inventory, Customers,

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,
- **Health:**
  - Telemedicine: MDLIVE, Doctor on Demand,
  - National: Aarogy Setu, CoWin, NACO App,
- **Organizational ERP: (Intranet)**
  - Institutions: Students, Faculty, Course
  - Manufacturing: Suppliers, Inventory, Customers,
  - Hospital: Patient, Doctor, OPD, IPD, Pharmacy,

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,
- **Health:**
  - Telemedicine: MDLIVE, Doctor on Demand,
  - National: Aarogy Setu, CoWin, NACO App,
- **Organizational ERP: (Intranet)**
  - Institutions: Students, Faculty, Course
  - Manufacturing: Suppliers, Inventory, Customers,
  - Hospital: Patient, Doctor, OPD, IPD, Pharmacy,
  - Bank: Customers, Accounts, Locker, Deposits,

Application Programs: Internet / Web or Mobile (cont.)

- **Document Processing:**
  - Editing: Overleaf, Google Docs, Spreadsheet
  - Website, Blog: Google Sites, WordPress, Webly,
- **Health:**
  - Telemedicine: MDLIVE, Doctor on Demand,
  - National: Aarogy Setu, CoWin, NACO App,
- **Organizational ERP: (Intranet)**
  - Institutions: Students, Faculty, Course
  - Manufacturing: Suppliers, Inventory, Customers,
  - Hospital: Patient, Doctor, OPD, IPD, Pharmacy,
  - Bank: Customers, Accounts, Locker, Deposits,
  - Courier: Customers, Parcels, Delivery Agents,

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data
  - Applications are functionally split into **frontend layer**, **middle layer**, **backend layer**

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data
  - Applications are functionally split into **frontend layer**, **middle layer**, **backend layer**
- **Frontend** or Presentation Layer / Tier

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data
  - Applications are functionally split into **frontend layer**, **middle layer**, **backend layer**
- **Frontend** or Presentation Layer / Tier
  - Interacts with the user: Display / View, Input / Output

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data
  - Applications are functionally split into **frontend layer**, **middle layer**, **backend layer**
- **Frontend** or Presentation Layer / Tier
  - Interacts with the user: Display / View, Input / Output
  - Choose item, Add to cart, Checkout, Pay, Track order

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data
  - Applications are functionally split into **frontend layer**, **middle layer**, **backend layer**
- **Frontend** or Presentation Layer / Tier
  - Interacts with the user: Display / View, Input / Output
  - Choose item, Add to cart, Checkout, Pay, Track order
  - Interfaces may be, Browser-based, Mobile App, or Custom

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data
  - Applications are functionally split into **frontend layer**, **middle layer**, **backend layer**
- **Frontend** or Presentation Layer / Tier
  - Interacts with the user: Display / View, Input / Output
  - Choose item, Add to cart, Checkout, Pay, Track order
  - Interfaces may be, Browser-based, Mobile App, or Custom
- **Middle** or Application / Business Logic Layer / Tier

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data
  - Applications are functionally split into **frontend layer**, **middle layer**, **backend layer**
- **Frontend** or Presentation Layer / Tier
  - Interacts with the user: Display / View, Input / Output
  - Choose item, Add to cart, Checkout, Pay, Track order
  - Interfaces may be, Browser-based, Mobile App, or Custom
- **Middle** or Application / Business Logic Layer / Tier
  - Implements the Functionality of the Application: Links front and backend

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data
  - Applications are functionally split into **frontend layer**, **middle layer**, **backend layer**
- **Frontend** or Presentation Layer / Tier
  - Interacts with the user: Display / View, Input / Output
  - Choose item, Add to cart, Checkout, Pay, Track order
  - Interfaces may be, Browser-based, Mobile App, or Custom
- **Middle** or Application / Business Logic Layer / Tier
  - Implements the Functionality of the Application: Links front and backend
  - Authentication, Search / Browse logic, Pricing, Cart management, Payment handling (gateway), Order management (mail / SMS / internal actions), Delivery management

Characteristic of Application Programs

- **Diversity**: These applications widely differ in their
  - Domain, functionality, user base, response time, scale, daily hit and many more
- **Unity**: Yet, these have a lot in common
  - Most use an RDBMS like Oracle, DB2 MySQL, PostgreSQL, etc. for managing data
  - Applications are functionally split into **frontend layer**, **middle layer**, **backend layer**
- **Frontend** or Presentation Layer / Tier
  - Interacts with the user: Display / View, Input / Output
  - Choose item, Add to cart, Checkout, Pay, Track order
  - Interfaces may be, Browser-based, Mobile App, or Custom
- **Middle** or Application / Business Logic Layer / Tier
  - Implements the Functionality of the Application: Links front and backend
  - Authentication, Search / Browse logic, Pricing, Cart management, Payment handling (gateway), Order management (mail / SMS / internal actions), Delivery management
  - Support functionality based on frontend interface

Characteristic of Application Programs (cont.)

- **Backend** or Data Access Layer / Tier

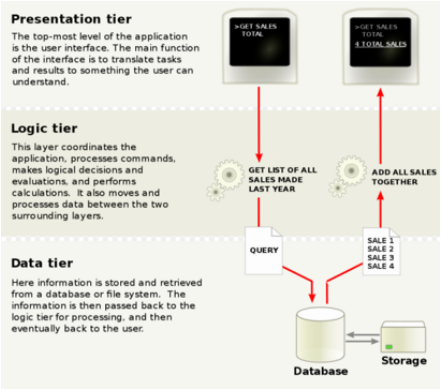
Characteristic of Application Programs (cont.)

- **Backend** or Data Access Layer / Tier
  - Manages persistent data, large volume, efficient access, security

Characteristic of Application Programs (cont.)

- **Backend** or Data Access Layer / Tier
  - Manages persistent data, large volume, efficient access, security
  - User, Cart, Inventory, Order, Vendor databases

Characteristic of Application Programs (2): Architecture



Source: https://en.wikipedia.org/wiki/Multitier_architecture

Application Architectures: Layers

- **Presentation Layer / Tier**

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device
  - **controller**: receives events, executes actions, and returns a view to the user

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device
  - **controller**: receives events, executes actions, and returns a view to the user
- **Business Logic Layer / Tier**

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device
  - **controller**: receives events, executes actions, and returns a view to the user
- **Business Logic Layer / Tier**
  - provides high level view of data and actions on data

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device
  - **controller**: receives events, executes actions, and returns a view to the user
- **Business Logic Layer / Tier**
  - provides high level view of data and actions on data
  - often using an object data model

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device
  - **controller**: receives events, executes actions, and returns a view to the user
- **Business Logic Layer / Tier**
  - provides high level view of data and actions on data
  - often using an object data model
  - hides details of data storage schema

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device
  - **controller**: receives events, executes actions, and returns a view to the user
- **Business Logic Layer / Tier**
  - provides high level view of data and actions on data
  - often using an object data model
  - hides details of data storage schema
- **Data Access Layer / Tier**

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device
  - **controller**: receives events, executes actions, and returns a view to the user
- **Business Logic Layer / Tier**
  - provides high level view of data and actions on data
  - often using an object data model
  - hides details of data storage schema
- **Data Access Layer / Tier**
  - interfaces between business logic layer and the underlying database

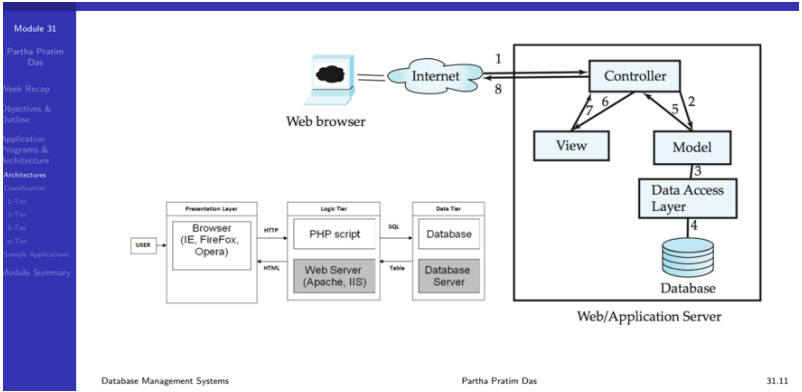
Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device
  - **controller**: receives events, executes actions, and returns a view to the user
- **Business Logic Layer / Tier**
  - provides high level view of data and actions on data
  - often using an object data model
  - hides details of data storage schema
- **Data Access Layer / Tier**
  - interfaces between business logic layer and the underlying database
  - provides mapping from object model of business layer to relational model of database

Application Architectures: Layers

- **Presentation Layer / Tier**
  - Model-View-Controller (MVC) architecture
  - **model**: business logic
  - **view**: presentation of data, depends on display device
  - **controller**: receives events, executes actions, and returns a view to the user
- **Business Logic Layer / Tier**
  - provides high level view of data and actions on data
  - often using an object data model
  - hides details of data storage schema
- **Data Access Layer / Tier**
  - interfaces between business logic layer and the underlying database
  - provides mapping from object model of business layer to relational model of database
  - Already discussed and studied in depth

Application Architecture (2): MVC



Application Architecture (3): User Interface

- Web browsers have become the de-facto standard user interface to databases

Application Architecture (3): User Interface

- Web browsers have become the de-facto standard user interface to databases
- Enable large numbers of users to access databases from anywhere

Application Architecture (3): User Interface

- Web browsers have become the de-facto standard user interface to databases
- Enable large numbers of users to access databases from anywhere
- Avoid the need for downloading / installing specialized code, while providing a good graphical user interface

Application Architecture (3): User Interface

- Web browsers have become the de-facto standard user interface to databases
- Enable large numbers of users to access databases from anywhere
- Avoid the need for downloading / installing specialized code, while providing a good graphical user interface
  - Javascript, Flash and other scripting languages run in browser, but are downloaded transparently

Application Architecture (3): User Interface

- Web browsers have become the de-facto standard user interface to databases
- Enable large numbers of users to access databases from anywhere
- Avoid the need for downloading / installing specialized code, while providing a good graphical user interface
  - Javascript, Flash and other scripting languages run in browser, but are downloaded transparently
- Examples: banks, airline and rental car reservations, university course registration and grading, and so on.

Application Architecture (3): User Interface

- Web browsers have become the de-facto standard user interface to databases
- Enable large numbers of users to access databases from anywhere
- Avoid the need for downloading / installing specialized code, while providing a good graphical user interface
  - Javascript, Flash and other scripting languages run in browser, but are downloaded transparently
- Examples: banks, airline and rental car reservations, university course registration and grading, and so on.
- Use in Mobile Devices are getting popular

Application Architecture (3): User Interface

- Web browsers have become the de-facto standard user interface to databases
- Enable large numbers of users to access databases from anywhere
- Avoid the need for downloading / installing specialized code, while providing a good graphical user interface
  - Javascript, Flash and other scripting languages run in browser, but are downloaded transparently
- Examples: banks, airline and rental car reservations, university course registration and grading, and so on.
- Use in Mobile Devices are getting popular
  - Mobile Apps or Browser in Mobile

Application Architecture (3): User Interface

- Web browsers have become the de-facto standard user interface to databases
- Enable large numbers of users to access databases from anywhere
- Avoid the need for downloading / installing specialized code, while providing a good graphical user interface
  - Javascript, Flash and other scripting languages run in browser, but are downloaded transparently
- Examples: banks, airline and rental car reservations, university course registration and grading, and so on.
- Use in Mobile Devices are getting popular
  - Mobile Apps or Browser in Mobile
- These are similar in architecture and workflow with web, but have significant differences with their smaller (but wide range of) form factor, and extremely low resources

Application Architecture (3): User Interface

- Web browsers have become the de-facto standard user interface to databases
- Enable large numbers of users to access databases from anywhere
- Avoid the need for downloading / installing specialized code, while providing a good graphical user interface
  - Javascript, Flash and other scripting languages run in browser, but are downloaded transparently
- Examples: banks, airline and rental car reservations, university course registration and grading, and so on.
- Use in Mobile Devices are getting popular
  - Mobile Apps or Browser in Mobile
- These are similar in architecture and workflow with web, but have significant differences with their smaller (but wide range of) form factor, and extremely low resources
- Will be discussed later

Application Architecture (4): Business Logic Layer

- Provides abstractions of entities

Application Architecture (4): Business Logic Layer

- Provides abstractions of entities
  - For example, students, instructors, courses, etc

Application Architecture (4): Business Logic Layer

- Provides abstractions of entities
  - For example, students, instructors, courses, etc
- Enforces business rules for carrying out actions

Application Architecture (4): Business Logic Layer

- Provides abstractions of entities
  - For example, students, instructors, courses, etc
- Enforces business rules for carrying out actions
  - For example, student can enroll in a class only if she has completed prerequisites, and has paid her tuition fees

Application Architecture (4): Business Logic Layer

- Provides abstractions of entities
  - For example, students, instructors, courses, etc
- Enforces business rules for carrying out actions
  - For example, student can enroll in a class only if she has completed prerequisites, and has paid her tuition fees
- Supports workflows which define how a task involving multiple participants is to be carried out

Application Architecture (4): Business Logic Layer

- Provides abstractions of entities
  - For example, students, instructors, courses, etc
- Enforces business rules for carrying out actions
  - For example, student can enroll in a class only if she has completed prerequisites, and has paid her tuition fees
- Supports workflows which define how a task involving multiple participants is to be carried out
  - For example, how to process application by a student applying to a university

Application Architecture (4): Business Logic Layer

- Provides abstractions of entities
  - For example, students, instructors, courses, etc
- Enforces business rules for carrying out actions
  - For example, student can enroll in a class only if she has completed prerequisites, and has paid her tuition fees
- Supports workflows which define how a task involving multiple participants is to be carried out
  - For example, how to process application by a student applying to a university
  - Sequence of steps to carry out task

Application Architecture (4): Business Logic Layer

- Provides abstractions of entities
  - For example, students, instructors, courses, etc
- Enforces business rules for carrying out actions
  - For example, student can enroll in a class only if she has completed prerequisites, and has paid her tuition fees
- Supports workflows which define how a task involving multiple participants is to be carried out
  - For example, how to process application by a student applying to a university
  - Sequence of steps to carry out task
  - Error handling

Application Architecture (4): Business Logic Layer

- Provides abstractions of entities
  - For example, students, instructors, courses, etc
- Enforces business rules for carrying out actions
  - For example, student can enroll in a class only if she has completed prerequisites, and has paid her tuition fees
- Supports workflows which define how a task involving multiple participants is to be carried out
  - For example, how to process application by a student applying to a university
  - Sequence of steps to carry out task
  - Error handling
  - For example, what to do if recommendation letters not received on time

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database
- alternative: implement object-oriented or object-relational database to store object model

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database
- alternative: implement object-oriented or object-relational database to store object model
  - has not been commercially successful

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database
- alternative: implement object-oriented or object-relational database to store object model
  - has not been commercially successful
- Schema designer has to provide a mapping between object data and relational schema

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database
- alternative: implement object-oriented or object-relational database to store object model
  - has not been commercially successful
- Schema designer has to provide a mapping between object data and relational schema
  - For example, Java class Student mapped to relation student, with corresponding mapping of attributes

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database
- alternative: implement object-oriented or object-relational database to store object model
  - has not been commercially successful
- Schema designer has to provide a mapping between object data and relational schema
  - For example, Java class Student mapped to relation student, with corresponding mapping of attributes
  - An object can map to multiple tuples in multiple relations

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database
- alternative: implement object-oriented or object-relational database to store object model
  - has not been commercially successful
- Schema designer has to provide a mapping between object data and relational schema
  - For example, Java class Student mapped to relation student, with corresponding mapping of attributes
  - An object can map to multiple tuples in multiple relations
- Application opens a session, which connects to the database

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database
- alternative: implement object-oriented or object-relational database to store object model
  - has not been commercially successful
- Schema designer has to provide a mapping between object data and relational schema
  - For example, Java class Student mapped to relation student, with corresponding mapping of attributes
  - An object can map to multiple tuples in multiple relations
- Application opens a session, which connects to the database
  - Objects can be created and saved to the database using session.save(object)

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database
- alternative: implement object-oriented or object-relational database to store object model
  - has not been commercially successful
- Schema designer has to provide a mapping between object data and relational schema
  - For example, Java class Student mapped to relation student, with corresponding mapping of attributes
  - An object can map to multiple tuples in multiple relations
- Application opens a session, which connects to the database
  - Objects can be created and saved to the database using session.save(object)
  - mapping used to create appropriate tuples in the database

Application Architecture (5): Object-Relational Mapping

- Allows application code to be written on top of object-oriented data model, while storing data in a traditional relational database
- alternative: implement object-oriented or object-relational database to store object model
  - has not been commercially successful
- Schema designer has to provide a mapping between object data and relational schema
  - For example, Java class Student mapped to relation student, with corresponding mapping of attributes
  - An object can map to multiple tuples in multiple relations
- Application opens a session, which connects to the database
  - Objects can be created and saved to the database using session.save(object)
  - mapping used to create appropriate tuples in the database
- Query can be run to retrieve objects satisfying specified predicates

Application Architecture (6): Data Access Layer

- Issues of modeling and design of databases have already discussed in depth through the previous module
- Issues of accessing and updating data from application will be discussed later this with through the interactions of native languages and SQL

Architecture Classification

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.
- The design of a DBMS depends on its architecture. It can be

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.
- The design of a DBMS depends on its architecture. It can be
  - centralized

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.
- The design of a DBMS depends on its architecture. It can be
  - centralized
  - decentralized

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.
- The design of a DBMS depends on its architecture. It can be
  - centralized
  - decentralized
  - hierarchical

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.
- The design of a DBMS depends on its architecture. It can be
  - centralized
  - decentralized
  - hierarchical
- The architecture of a DBMS can be seen as either single tier or multi-tier:

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.
- The design of a DBMS depends on its architecture. It can be
  - centralized
  - decentralized
  - hierarchical
- The architecture of a DBMS can be seen as either single tier or multi-tier:
  - 1-tier architecture

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.
- The design of a DBMS depends on its architecture. It can be
  - centralized
  - decentralized
  - hierarchical
- The architecture of a DBMS can be seen as either single tier or multi-tier:
  - 1-tier architecture
  - 2-tier architecture

Architecture Classification

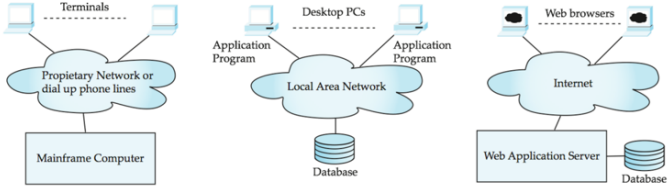
- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.
- The design of a DBMS depends on its architecture. It can be
  - centralized
  - decentralized
  - hierarchical
- The architecture of a DBMS can be seen as either single tier or multi-tier:
  - 1-tier architecture
  - 2-tier architecture
  - 3-tier architecture

Architecture Classification

- Database architecture uses programming languages to design a particular type of software for businesses or organizations.
- Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions.
- A database architect develops and implements software to meet the needs of users.
- The design of a DBMS depends on its architecture. It can be
  - centralized
  - decentralized
  - hierarchical
- The architecture of a DBMS can be seen as either single tier or multi-tier:
  - 1-tier architecture
  - 2-tier architecture
  - 3-tier architecture
  - n-tier architecture

Architecture Evolution

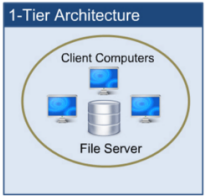
- Three distinct eras of application architecture
  - Mainframe (1960's and 70's)
  - Personal computer era (1980's)
  - Web / Mobile era (1990's onwards)



(a) Mainframe Era (b) Personal Computer Era (c) Web era

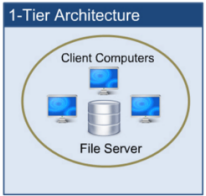
1-tier Architecture

- One-tier architecture involves putting all of the required components for a software application or technology on a single server or platform



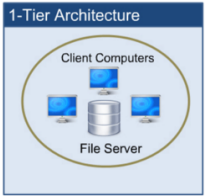
1-tier Architecture

- One-tier architecture involves putting all of the required components for a software application or technology on a single server or platform
- Basically, a one-tier architecture keeps all of the elements of an application, including the interface, Middleware and back-end data, in one place



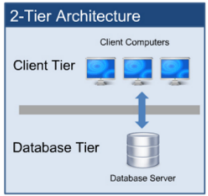
1-tier Architecture

- One-tier architecture involves putting all of the required components for a software application or technology on a single server or platform
- Basically, a one-tier architecture keeps all of the elements of an application, including the interface, Middleware and back-end data, in one place
- Developers see these types of systems as the simplest and most direct way



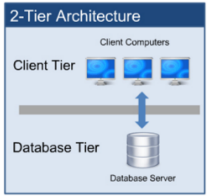
2-tier Architecture

- The two-tier is based on Client Server architecture



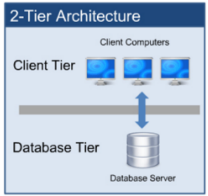
2-tier Architecture

- The two-tier is based on Client Server architecture
- It is like client server application



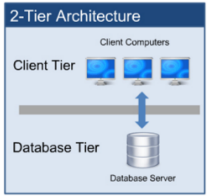
2-tier Architecture

- The two-tier is based on Client Server architecture
- It is like client server application
- The direct communication takes place between client and server



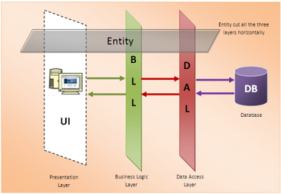
2-tier Architecture

- The two-tier is based on Client Server architecture
- It is like client server application
- The direct communication takes place between client and server
- There is no intermediate between client and server



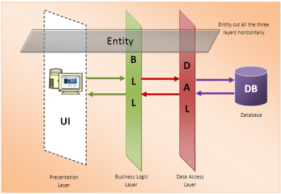
3-tier Architecture

- A 3-tier architecture separates its tiers Presentation, Logic and Data Access - from each other based on the complexity of the users and how they use the data present in the database



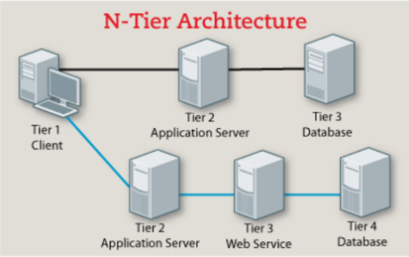
3-tier Architecture

- A 3-tier architecture separates its tiers Presentation, Logic and Data Access - from each other based on the complexity of the users and how they use the data present in the database
- It is the most widely used architecture to design a DBMS



n-tier Architecture

- An n-tier architecture distributes different components of the 3 tiers between different servers and adds interfaces tiers for interactions and workload balancing



Sample Applications in Multiple Tiers

Application	Presentation	Logic	Data	Functionality
Web Mail	Login Mail List View Inbox Sent Items Outbox Trash Mail Composer Filters	User Authentication Connection to Mail Server (SMTP, POP, IMAP) Encryption Decryption	Mail Users Address Book Mail Items	Send Receive Mails Manage Address Book
Net Banking	Login Account View Add Delete Account Add Delete Beneficiary Fund Transfer	User Authentication Beneficiary Authentication Transaction Validation Connection to Banks Gateways Encryption Decryption	Account Holders Beneficiaries Accounts Debit Credit Transactions	Check Balance and Transactions Transfer Funds
Timetable	Login Add Delete Courses, Teachers, Rooms, Slots Assignments: Teachers Course Allocations Course Room, Slots	User Authentication Timetable Assignment Logic Encryption Decryption	Courses Teachers Rooms Slots Assignments Allocations	Manage timetable for multiple courses taken by multiple teachers

Module Summary

Summary

- Had a glimpse of Application Programs across various sectors

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- Had a glimpse of Application Programs across various sectors
- Understood the typical architecture for an application

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- Had a glimpse of Application Programs across various sectors
- Understood the typical architecture for an application
- Studies the classification and evolution of the architectures

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- Had a glimpse of Application Programs across various sectors
- Understood the typical architecture for an application
- Studies the classification and evolution of the architectures
- Glimpsed at architecture for a few sample applications

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.